COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 212 - Automating Server Management		**SECTION:**	CPE31S2
DATE PERFORMED:	10 - 06 - 26		**DATE SUBMITTED:**	10 - 13 - 26
NAME:	Santiago, David Owen A.		**INSTRUCTOR:**	Engr. Jimlord Quejado

Activity 10.1	**Install, Configure, and Manage Log Monitoring tools**

1. PROCEDURE OUTPUT

Create a playbook that:
Install and configure Elastic Stack in separate hosts (Elastic Search, Kibana, Logstash)

INSTALLATION STEPS:

1. Create separate roles for each task:

```
santiago@workstation:~/CPE212-Santiago/ansible-lab/plays$ cd ..
santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible-galaxy init elastic_search
- Role elastic_search was created successfully
santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible-galaxy init kibana
- Role kibana was created successfully
santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible-galaxy init logstash
- Role logstash was created successfully
santiago@workstation:~/CPE212-Santiago/ansible-lab$
```

2. Create group_vars directory to define ELK shared variables and password vault:

```
santiago@workstation:~/CPE212-Santiago/ansible-lab$ tree gr*
group_vars
└── all
    ├── main.yml
    └── vault.yml
```

3. Define which ELK component will be installed in elk host

Elastic Search: web_servers | ubuntu1 and centos1
Kibana: db_servers | ubuntu2
Logstash: workstations | ubuntu1

```
web_servers: # Elasticsearch
  hosts:
    ubuntu1:
      ansible_host: 192.168.56.102
      ansible_user: server1
    centos1:
      ansible_host: 192.168.56.104
      ansible_user: santiago
```

```
db_servers: # Kibana
  hosts:
    ubuntu2:
      ansible_host: 192.168.56.103
      ansible_user: server2
```

```
workstations: # Logstash
  hosts:
    ubuntu1:
      ansible_host: 192.168.56.102
      ansible_user: server1
```

4. Define shared variables

Define the GPG key and keyring path in this playbook:

```
 GNU nano 8.7.1                                    main.yml *
---
elastic_series: "9.x"
elastic_gpg_key_url: https://artifacts.elastic.co/GPG-KEY-elasticsearch
elastic_keyring_path: /usr/share/keyrings/elasticsearch-keyring.gpg

# Repositories (APT and YUM)
elastic_apt_repo: "deb [signed-by={{ elastic_keyring_path }}] https://artifacts.elasti
elastic_yum_baseurl: "https://artifacts.elastic.co/packages/{{ elastic_series }}/yum"
```

Since web_servers will be the first host to store elastic, it will be the primary host group. Kibana will be enrolled into this instance of elastic.

```
# First Elasticsearch host; Kibana and Logstash connect to it
es_primary_host: "{{ groups['web_servers'][0] }}"
es_primary_addr: "{{ hostvars[es_primary_host].ansible_host }}"
es_elastic_password: "{{ vault_elastic_password }}"
```

Elastic password:

```
  GNU nano 8.7.1
---
vault_elastic_password: "santiagoelastic"
```

Encrypting:

```
santiago@workstation:~/CPE212-Santiago/ansible-lab/group_vars/all$ ansible-vault encrypt vault.yml
New Vault password:
Confirm New Vault password:
Encryption successful
```

NOTE: Not all components will be installed to the CentOS node but I added install_redhat.yml to all roles in the chance that they are also installed in CentOS.

5. Elastic Search role
   a. Main Playbook (elasticsearch.yml)

```
  GNU nano 8.7.1
---
- name: Install and configure Elasticsearch
  hosts: web_servers
  become: true
  roles:
    - elasticsearch
```

   b. Defaults

```
  GNU nano 8.7.1                                    main.yml
#SPDX-License-Identifier: MIT-0
---
# defaults file for elastic_search
es_cluster_name: elk-lab
es_node_name: "{{ ansible_hostname }}"
es_heap_size: 1g          # Xms and Xmx must match; keep small for lab VMs
es_http_host: 0.0.0.0     # lets Kibana and Logstash reach the HTTP API
es_http_port: 9200
es_http_publish_host: "{{ ansible_host }}"
```

c. Tasks

main.yml:

```
  GNU nano 8.7.1                                                main.yml
#SPDX-License-Identifier: MIT-0
---
# OS resolves to either "Debian" or "RedHat"
- name: Include OS-specific install tasks
  ansible.builtin.include_tasks: "install_{{ ansible_os_family | lower }}.yml"

- name: Configure Elasticsearch
  ansible.builtin.include_tasks: configure.yml

- name: Start Elasticsearch and set the elastic password
  ansible.builtin.include_tasks: security.yml

- name: Open the firewall for the HTTP port
  ansible.builtin.include_tasks: firewall.yml
```

firewall.yml

```
  GNU nano 8.7.1                                                firewall.ym
---
# Let Kibana and Logstash on other hosts reach the HTTP API.
# Only needed when a host firewall is active.

- name: Open the Elasticsearch HTTP port (ufw)
  community.general.ufw:
    rule: allow
    port: "{{ es_http_port }}"
    proto: tcp
  when: ansible_os_family == "Debian"

- name: Open the Elasticsearch HTTP port (firewalld)
  ansible.posix.firewalld:
    port: "{{ es_http_port }}/tcp"
    permanent: true
    immediate: true
    state: enabled
  when: ansible_os_family == "RedHat"
```

install_debian.yml:

```
  GNU nano 8.7.1                                                install_debi
---
- name: Install prerequisites
  ansible.builtin.apt:
    name: [apt-transport-https, gnupg, wget]
    state: present
    update_cache: true
# The key is fetched and dearmored into /usr/share/keyrings.
- name: Download the Elastic GPG key
  ansible.builtin.get_url:
    url: "{{ elastic_gpg_key_url }}"
    dest: /tmp/GPG-KEY-elasticsearch
    mode: "0644"

- name: Convert the key to a keyring (gpg --dearmor)
  ansible.builtin.command: >
    gpg --dearmor -o {{ elastic_keyring_path }} /tmp/GPG-KEY-elasticsearch
  args:
    creates: "{{ elastic_keyring_path }}"

# The repo is written to /etc/apt/sources.list.d/elastic-9.x.list.
- name: Add the Elastic APT repository
  ansible.builtin.apt_repository:
    repo: "{{ elastic_apt_repo }}"
    filename: elastic-{{ elastic_series }}
    state: present

- name: Install the Elasticsearch package
  ansible.builtin.apt:
    name: elasticsearch
    state: present
    update_cache: true
```

install_redhat.yml:

```
  GNU nano 8.7.1                                          install_redhat.
---
- name: Import the Elastic GPG key
  ansible.builtin.rpm_key:
    key: "{{ elastic_gpg_key_url }}"
    state: present

- name: Define the Elasticsearch yum repository
  ansible.builtin.yum_repository:
    name: elasticsearch
    file: elasticsearch
    description: "Elasticsearch repository for {{ elastic_series }} packages"
    baseurl: "{{ elastic_yum_baseurl }}"
    gpgcheck: true
    gpgkey: "{{ elastic_gpg_key_url }}"
    enabled: false          # the RPM docs ship this repo disabled
    autorefresh: true
    type: rpm-md

- name: Install Elasticsearch from the repo
  ansible.builtin.yum:
    name: elasticsearch
    state: present
    enablerepo: elasticsearch
```

NOTE: dnf and yum works either way, I used yum because the documentation uses yum.

configure.yml:

```
---
# applies the settings from the "important settings" page:
# cluster name, node name, network, and heap.

- name: Set core settings in elasticsearch.yml
  ansible.builtin.lineinfile:
    path: /etc/elasticsearch/elasticsearch.yml
    regexp: "^#?{{ item.key }}:"
    line: "{{ item.key }}: {{ item.value }}"
  loop:
    - { key: "cluster.name", value: "{{ es_cluster_name }}" }
    - { key: "node.name",    value: "{{ es_node_name }}" }
    - { key: "http.host",    value: "{{ es_http_host }}" }
    - { key: "http.port",    value: "{{ es_http_port }}" }
    - { key: "http.publish_host", value: "{{ es_http_publish_host }}" }
  notify: Restart elasticsearch
```

These steps were fetched from the official configuration. The last key pair ensures that the hostname will be taken as the ansible_host (hostname

server1) of the actual machine and not what it's named in the inventory (ubuntu1).

```
- name: Set JVM heap size
  ansible.builtin.template:
    src: heap.options.j2
    dest: /etc/elasticsearch/jvm.options.d/heap.options
    owner: root
    group: elasticsearch
    mode: "0660"
  notify: Restart elasticsearch
```

```
- name: Apply config changes before continuing
  ansible.builtin.meta: flush_handlers
```

The jinja2 file contains the heap configuration.

security.yml
Enables and starts elastic search then generates random elastic password.

```
  GNU nano 8.7.1                                        security.yml *
---
# On first start, 9.x auto-configures security and generates a random elastic password.
# The reset task sets a known password so Kibana and Logstash can use it.
# The marker file (creates) keeps the task idempotent.
# The final check is the documented "curl to port 9200" test.

- name: Reload systemd, enable and start Elasticsearch
  ansible.builtin.systemd:
    name: elasticsearch
    daemon_reload: true
    enabled: true
    state: started

- name: Wait for the HTTP port
  ansible.builtin.wait_for:
    port: "{{ es_http_port }}"
    timeout: 180

- name: Generate a temporary elastic password (first run only)
  ansible.builtin.command: >
    /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic -a -b -s
  args:
    creates: /etc/elasticsearch/.elastic_password_set
  register: es_temp_pw
  retries: 6
  delay: 10
  until: es_temp_pw.rc == 0
  no_log: "{{ es_no_log | default(true) }}"
```

```yaml
- name: Set the elastic password to the vaulted value
  ansible.builtin.uri:
    url: "https://localhost:{{ es_http_port }}/_security/user/elastic/_password"
    method: POST
    user: elastic
    password: "{{ es_temp_pw.stdout_lines | last | trim }}"
    force_basic_auth: true
    body_format: json
    body:
      password: "{{ es_elastic_password }}"
    validate_certs: false
    status_code: 200
  when: es_temp_pw is changed
  no_log: "{{ es_no_log | default(true) }}"

- name: Mark the password as set
  ansible.builtin.file:
    path: /etc/elasticsearch/.elastic_password_set
    state: touch
    mode: "0600"
  when: es_temp_pw is changed

- name: Verify Elasticsearch responds
  ansible.builtin.uri:
    url: "https://localhost:{{ es_http_port }}"
    user: elastic
    password: "{{ es_elastic_password }}"
    force_basic_auth: true
    validate_certs: false      # self-signed auto-generated CA (lab only)
    status_code: 200
```

d. Templates

heap.options.j2:

```
  GNU nano 8.7.1
-Xms{{ es_heap_size }}
-Xmx{{ es_heap_size }}
```

e. Handlers

```
  GNU nano 8.7.1
#SPDX-License-Identifier: MIT-0
---
# handlers file for elastic_search
- name: Restart elasticsearch
  ansible.builtin.systemd:
    name: elasticsearch
    state: restarted
```

6. Kibana Role
   a. Main Playbook (kibana.yml)

```
  GNU nano 8.7.1
---
- name: Install and configure Kibana
  hosts: db_servers
  become: true
  roles:
    - kibana
```

   b. Defaults

ansible-lab > roles > kibana > defaults > ! main.yml

```
1   #SPDX-License-Identifier: MIT-0
2   ---
3   # defaults file for kibana
4   kibana_server_host: 0.0.0.0
5   kibana_server_port: 5601
6   kibana_server_name: "{{ ansible_hostname }}"
7   kibana_public_url: "http://{{ ansible_host }}:{{ kibana_server_port }}"
```

   c. Tasks

main.yml:

```
  GNU nano 8.7.1                                                main.yml
#SPDX-License-Identifier: MIT-0
---
# tasks file for kibana
- name: Include OS-specific install tasks
  ansible.builtin.include_tasks: "install_{{ ansible_os_family | lower }}.yml"

- name: Configure Kibana
  ansible.builtin.include_tasks: configure.yml

- name: Enroll Kibana with Elasticsearch
  ansible.builtin.include_tasks: enroll.yml

- name: Enable and start Kibana
  ansible.builtin.systemd:
    name: kibana
    daemon_reload: true
    enabled: true
    state: started

- name: Wait for Kibana to listen
  ansible.builtin.wait_for:
    port: "{{ kibana_server_port }}"
    timeout: 180
```

install_debian.yml:

```
 GNU nano 8.7.1                                                install_debian.yml
- name: Install prerequisites
  ansible.builtin.apt:
    name: [apt-transport-https, gnupg, wget]
    state: present
    update_cache: true
```

```
- name: Download the Elastic GPG key
  ansible.builtin.get_url:
    url: "{{ elastic_gpg_key_url }}"
    dest: /tmp/GPG-KEY-elasticsearch
    mode: "0644"

- name: Convert the key to a keyring
  ansible.builtin.command: gpg --dearmor -o {{ elastic_keyring_path }} /tmp/GPG-KEY-elasticsearch
  args:
    creates: "{{ elastic_keyring_path }}"

- name: Add the Elastic APT repository
  ansible.builtin.apt_repository:
    repo: "{{ elastic_apt_repo }}"
    filename: elastic-{{ elastic_series }}
    state: present

- name: Install Kibana
  ansible.builtin.apt:
    name: kibana
    state: present
    update_cache: true
```

install_redhat.yml:

```
  GNU nano 8.7.1                                                install_
---
- name: Import the Elastic GPG key
  ansible.builtin.rpm_key:
    key: "{{ elastic_gpg_key_url }}"
    state: present

- name: Define the Kibana yum repository
  ansible.builtin.yum_repository:
    name: "kibana-{{ elastic_series }}"
    file: kibana
    description: "Kibana repository for {{ elastic_series }} packages"
    baseurl: "{{ elastic_yum_baseurl }}"
    gpgcheck: true
    gpgkey: "{{ elastic_gpg_key_url }}"
    enabled: true
    autorefresh: true
    type: rpm-md

- name: Install Kibana
  ansible.builtin.yum:
    name: kibana
    state: present
```

configure.yml

```
  GNU nano 8.7.1                                                configure.
---
- name: Set Kibana server settings
  ansible.builtin.lineinfile:
    path: /etc/kibana/kibana.yml
    regexp: "^#?{{ item.key }}:"
    line: "{{ item.key }}: {{ item.value }}"
  loop:
    - { key: "server.port",          value: "{{ kibana_server_port }}" }
    - { key: "server.host",          value: '"{{ kibana_server_host }}"' }
    - { key: "server.name",          value: '"{{ kibana_server_name }}"' }
    - { key: "server.publicBaseUrl", value: '"{{ kibana_public_url }}"' }
  notify: Restart kibana
```

enroll.yml:

```
  GNU nano 8.7.1                                                enroll.yml
---
- name: Check whether Kibana is already enrolled
  ansible.builtin.command: grep -q '^elasticsearch.hosts' /etc/kibana/kibana.yml
  register: kibana_enrolled
  failed_when: kibana_enrolled.rc > 1
  changed_when: false

- name: Generate a Kibana enrollment token on the Elasticsearch host
  ansible.builtin.command: >
    /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana
  delegate_to: "{{ es_primary_host }}"
  register: kibana_token
  no_log: true
  when: kibana_enrolled.rc == 1

- name: Enroll Kibana using the token
  ansible.builtin.command: >
    /usr/share/kibana/bin/kibana-setup --enrollment-token {{ kibana_token.stdout }}
  no_log: true
  when: kibana_enrolled.rc == 1
  notify: Restart kibana

- name: Ensure Kibana can read its config
  ansible.builtin.file:
    path: /etc/kibana/kibana.yml
    owner: root
    group: kibana
    mode: "0660"
```

This enrolls this instance of Kibana to the existing elastic installation in the web_servers.

d. Handlers

```
  GNU nano 8.7.1
#SPDX-License-Identifier: MIT-0
---
# handlers file for kibana
- name: Restart kibana
  ansible.builtin.systemd:
    name: kibana
    state: restarted
```

7. Logstash Role
    a. Main Playbook (logstash.yml):

```
  GNU nano 8.7.1
---
- name: Install and configure Logstash
  hosts: workstations
  become: true
  roles:
    - logstash
```

b. Defaults

```
  GNU nano 8.7.1
#SPDX-License-Identifier: MIT-0
---
# defaults file for logstash
logstash_beats_port: 5044
logstash_ca_path: /etc/logstash/certs/http_ca.crt
logstash_index_pattern: "logstash-%{+YYYY.MM.dd}"
```

c. Tasks

main.yml:

```
  GNU nano 8.7.1                                          main.yml
#SPDX-License-Identifier: MIT-0
---
# tasks file for logstash
- name: Include OS-specific install tasks
  ansible.builtin.include_tasks: "install_{{ ansible_os_family | lower }}.yml"

- name: Configure Logstash pipeline
  ansible.builtin.include_tasks: configure.yml

- name: Enable and start Logstash
  ansible.builtin.systemd:
    name: logstash
    daemon_reload: true
    enabled: true
    state: started
```

install_debian.yml:

```
 GNU nano 8.7.1                                    install_debian.yml
---
- name: Install prerequisites
  ansible.builtin.apt:
    name: [apt-transport-https, gnupg, wget]
    state: present
    update_cache: true

- name: Download the Elastic GPG key
  ansible.builtin.get_url:
    url: "{{ elastic_gpg_key_url }}"
    dest: /tmp/GPG-KEY-elasticsearch
    mode: "0644"

- name: Convert the key to a keyring
  ansible.builtin.command: gpg --dearmor -o {{ elastic_keyring_path }} /tmp/GPG-KEY-elasticsearch
  args:
    creates: "{{ elastic_keyring_path }}"

- name: Add the Elastic APT repository
  ansible.builtin.apt_repository:
    repo: "{{ elastic_apt_repo }}"
    filename: elastic-{{ elastic_series }}
    state: present

- name: Install Logstash
  ansible.builtin.apt:
    name: logstash
    state: present
    update_cache: true
```

install_redhat.yml:

```
 GNU nano 8.7.1                                    install_redhat.yml
---
- name: Import the Elastic GPG key
  ansible.builtin.rpm_key:
    key: "{{ elastic_gpg_key_url }}"
    state: present

- name: Define the Logstash yum repository
  ansible.builtin.yum_repository:
    name: "logstash-{{ elastic_series }}"
    file: logstash
    description: "Elastic repository for {{ elastic_series }} packages"
    baseurl: "{{ elastic_yum_baseurl }}"
    gpgcheck: true
    gpgkey: "{{ elastic_gpg_key_url }}"
    enabled: true
    autorefresh: true
    type: rpm-md

- name: Install Logstash
  ansible.builtin.yum:
    name: logstash
    state: present
```

configure.yml:

GNU nano 8.7.1 configure.yml

```
---
- name: Read the Elasticsearch CA certificate from the ES host
  ansible.builtin.slurp:
    src: /etc/elasticsearch/certs/http_ca.crt
  delegate_to: "{{ es_primary_host }}"
  register: es_ca

- name: Create the certs directory
  ansible.builtin.file:
    path: /etc/logstash/certs
    state: directory
    owner: root
    group: logstash
    mode: "0750"

- name: Install the CA certificate for Logstash
  ansible.builtin.copy:
    content: "{{ es_ca.content | b64decode }}"
    dest: "{{ logstash_ca_path }}"
    owner: root
    group: logstash
    mode: "0640"
  notify: Restart logstash

- name: Deploy the Beats-to-Elasticsearch pipeline
  ansible.builtin.template:
    src: beats-to-es.conf.j2
    dest: /etc/logstash/conf.d/beats-to-es.conf
    owner: root
    group: logstash
    mode: "0640"          # the file contains a password
  notify: Restart logstash
```

d. Templates

beats-to-es.conf.j2:

```
  GNU nano 8.7.1                                          beats-to-es.conf.j2
input {
  beats {
    port => {{ logstash_beats_port }}
  }
}

output {
  elasticsearch {
    hosts => ["https://{{ es_primary_addr }}:9200"]
    user => "elastic"
    password => "{{ es_elastic_password }}"
    ssl_certificate_authorities => ["{{ logstash_ca_path }}"]
    index => "{{ logstash_index_pattern }}"
  }
}
```

e. Handlers

```
#SPDX-License-Identifier: MIT-0
---
# handlers file for logstash
- name: Restart logstash
  ansible.builtin.systemd:
    name: logstash
    state: restarted
```

8. ELK Site playbook (install_elk.yml)

install_elk_all.yml:

```
  GNU nano 8.7.1                                          install_elk_al
---

# Install according to dependency order

- import_playbook: elasticsearch.yml
- import_playbook: kibana.yml
- import_playbook: logstash.yml
```

RUNNING THE PLAYBOOKS:

Tests:
Check if ansible-vault works and for syntax errors:

```
santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible-playbook plays/install_elk_all.yml
 --ask-vault-pass --syntax-check
Vault password:

playbook: plays/install_elk_all.yml
santiago@workstation:~/CPE212-Santiago/ansible-lab$
```

Verify that correct hosts are assigned to each ELK Component:

```
santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible-playbook plays/install_elk_all.yml
 --ask-vault-pass --list-hosts
Vault password:

playbook: plays/install_elk_all.yml

  play #1 (web_servers): Install and configure Elasticsearch    TAGS: []
    pattern: ['web_servers']
    hosts (2):
      ubuntu1
      centos1

  play #2 (db_servers): Install and configure Kibana    TAGS: []
    pattern: ['db_servers']
    hosts (1):
      ubuntu2

  play #3 (workstations): Install and configure Logstash        TAGS: []
    pattern: ['workstations']
    hosts (1):
      ubuntu1
santiago@workstation:~/CPE212-Santiago/ansible-lab$
```

Verify task list:

```
santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible-playbook plays/install_elk_all.yml
 --ask-vault-pass --list-tasks
Vault password:

playbook: plays/install_elk_all.yml

  play #1 (web_servers): Install and configure Elasticsearch    TAGS: []
    tasks:
      elasticsearch : Include OS-specific install tasks TAGS: []
      elasticsearch : Configure Elasticsearch   TAGS: []
      elasticsearch : Start Elasticsearch and set the elastic password  TAGS: []

  play #2 (db_servers): Install and configure Kibana    TAGS: []
    tasks:
      kibana : Include OS-specific install tasks        TAGS: []
      kibana : Configure Kibana TAGS: []
      kibana : Enroll Kibana with Elasticsearch TAGS: []
      kibana : Enable and start Kibana  TAGS: []
      kibana : Wait for Kibana to listen        TAGS: []

  play #3 (workstations): Install and configure Logstash        TAGS: []
    tasks:
      logstash : Include OS-specific install tasks      TAGS: []
      logstash : Configure Logstash pipeline    TAGS: []
      logstash : Enable and start Logstash      TAGS: []
santiago@workstation:~/CPE212-Santiago/ansible-lab$
```

Run the playbook:

```
santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible-playbook plays/install_elk_all.yml --ask-vault-pass
BECOME password:
Vault password:

PLAY [Install and configure Elasticsearch] ************************************************************************

TASK [Gathering Facts] ********************************************************************************************
[WARNING]: Host 'ubuntu1' is using the discovered Python interpreter at '/usr/bin/python3.14', but future installation of ano
ther Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/
reference_appendices/interpreter_discovery.html for more information.
ok: [ubuntu1]
[WARNING]: Host 'centos1' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of anot
her Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/r
eference_appendices/interpreter_discovery.html for more information.
ok: [centos1]
```

```
TASK [elasticsearch : Define the Elasticsearch yum repository] ************************************************************
[ERROR]: Task failed: Module failed: Unsupported parameters for (ansible.builtin.yum_repository) module: autorefresh, type. S
upported parameters include: async, attributes, bandwidth, baseurl, cost, countme, deltarpm_metadata_percentage, deltarpm_per
centage, description, enabled, enablegroups, exclude, failovermethod, file, gpgcakey, gpgcheck, gpgkey, group, http_caching,
include, includepkgs, ip_resolve, keepalive, metadata_expire, metadata_expire_filter, metalink, mirrorlist, mirrorlist_expire
, mode, module_hotfixes, name, owner, password, priority, protect, proxy, proxy_password, proxy_username, repo_gpgcheck, repo
sdir, retries, s3_enabled, selevel, serole, setype, seuser, skip_if_unavailable, ssl_check_cert_permissions, sslcacert, sslcl
ientcert, sslclientkey, sslverify, state, throttle, timeout, ui_repoid_vars, unsafe_writes, username (attr, ca_cert, client_c
ert, client_key, excludepkgs, validate_certs).
Origin: /home/santiago/CPE212-Santiago/ansible-lab/roles/elasticsearch/tasks/install_redhat.yml:7:3

5     state: present
6
7 - name: Define the Elasticsearch yum repository
    ^ column 3

fatal: [centos1]: FAILED! => {"changed": false, "msg": "Unsupported parameters for (ansible.builtin.yum_repository) module: a
utorefresh, type. Supported parameters include: async, attributes, bandwidth, baseurl, cost, countme, deltarpm_metadata_perce
ntage, deltarpm_percentage, description, enabled, enablegroups, exclude, failovermethod, file, gpgcakey, gpgcheck, gpgkey, gr
oup, http_caching, include, includepkgs, ip_resolve, keepalive, metadata_expire, metadata_expire_filter, metalink, mirrorlist
, mirrorlist_expire, mode, module_hotfixes, name, owner, password, priority, protect, proxy, proxy_password, proxy_username,
repo_gpgcheck, reposdir, retries, s3_enabled, selevel, serole, setype, seuser, skip_if_unavailable, ssl_check_cert_permission
s, sslcacert, sslclientcert, sslclientkey, sslverify, state, throttle, timeout, ui_repoid_vars, unsafe_writes, username (attr
, ca_cert, client_cert, client_key, excludepkgs, validate_certs)."}
```

Error because the ansible.builtin.yum_repository has no autorefresh or type parameter.
Fix: Removing these two lines from the install_redhat playbooks in each role

```
- name: Define the Logstash yum repository
  ansible.builtin.yum_repository:
    name: "logstash-{{ elastic_series }}"
    file: logstash
    description: "Elastic repository for {{ elast
    baseurl: "{{ elastic_yum_baseurl }}"
    gpgcheck: true
    gpgkey: "{{ elastic_gpg_key_url }}"
    enabled: true
    autorefresh: true
    type: rpm-md
```

Running the playbook again:

```
TASK [elasticsearch : Define the Elasticsearch yum repository] ************************************************************
changed: [centos1]

TASK [elasticsearch : Install Elasticsearch from the repo] ****************************************************************
```

Error was fixed

FINAL PLAYBOOK OUTPUT:

```
TASK [logstash : Create the certs directory] ************************************************************************
changed: [ubuntu1]

TASK [logstash : Install the CA certificate for Logstash] ***********************************************************
changed: [ubuntu1]

TASK [logstash : Deploy the Beats-to-Elasticsearch pipeline] ********************************************************
changed: [ubuntu1]

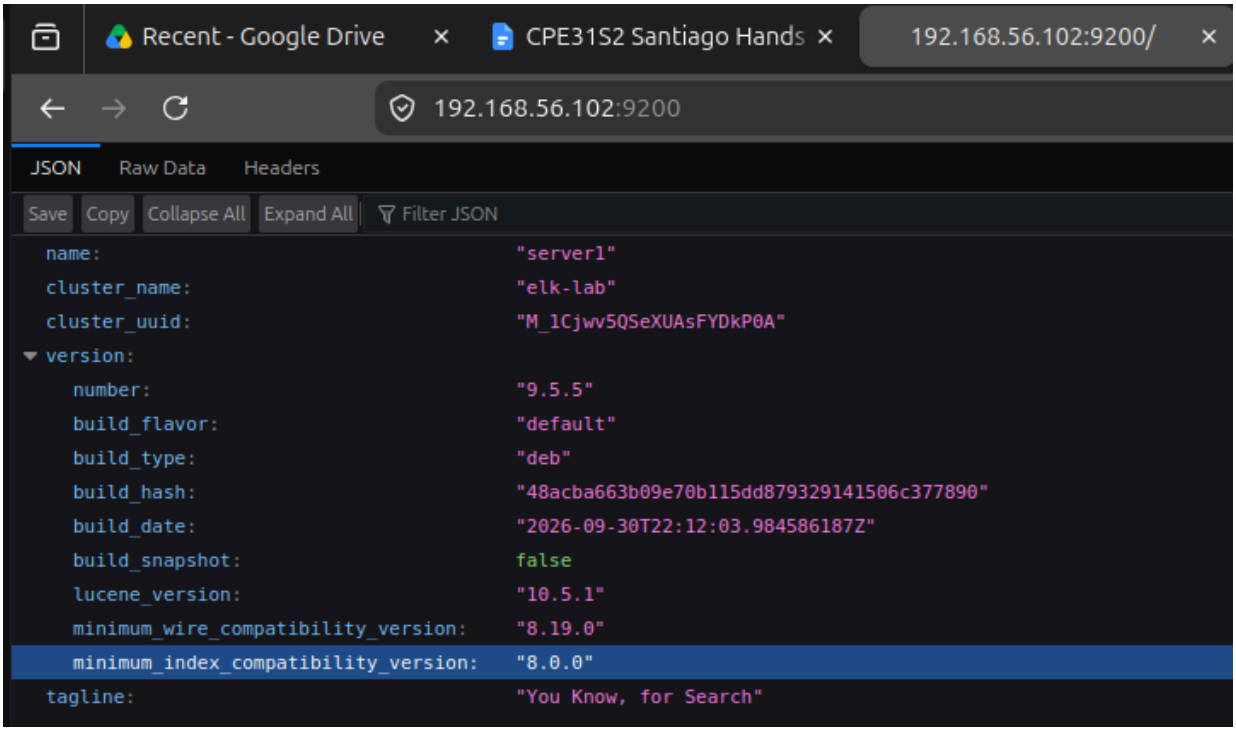
TASK [logstash : Enable and start Logstash] *************************************************************************
changed: [ubuntu1]

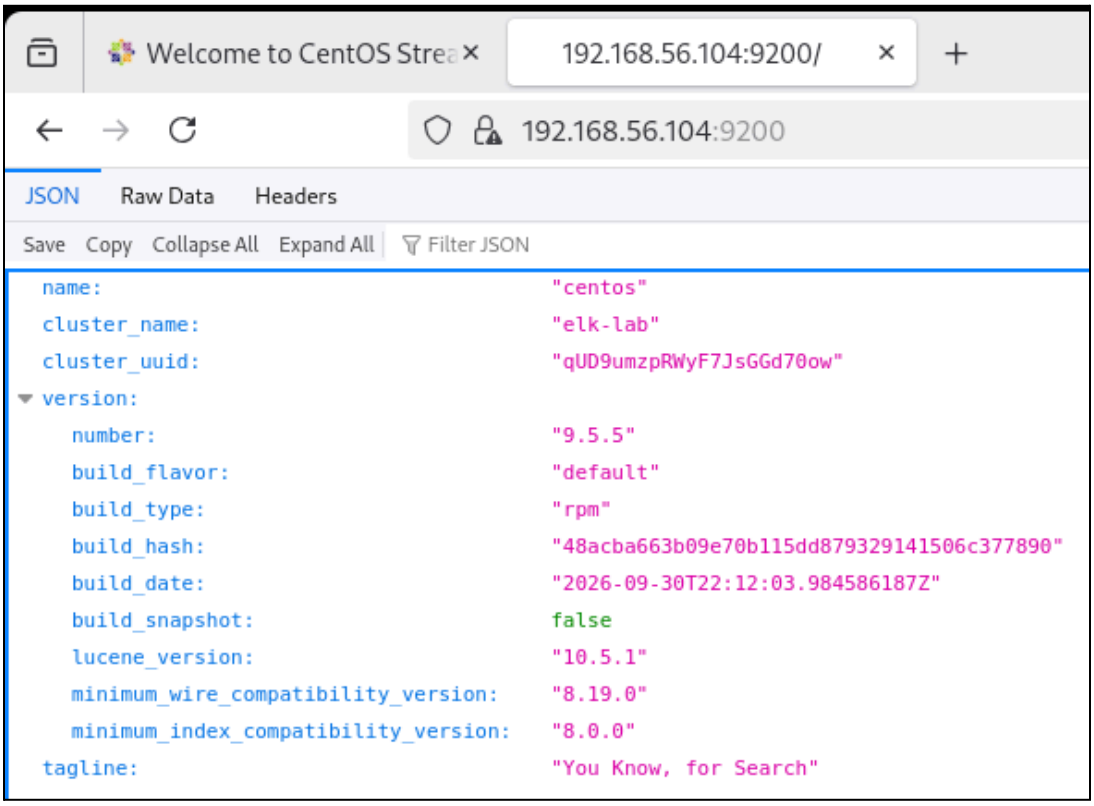
RUNNING HANDLER [logstash : Restart logstash] ***********************************************************************
changed: [ubuntu1]

PLAY RECAP **********************************************************************************************************
centos1                    : ok=15   changed=1    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
ubuntu1                    : ok=31   changed=7    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
ubuntu2                    : ok=17   changed=4    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

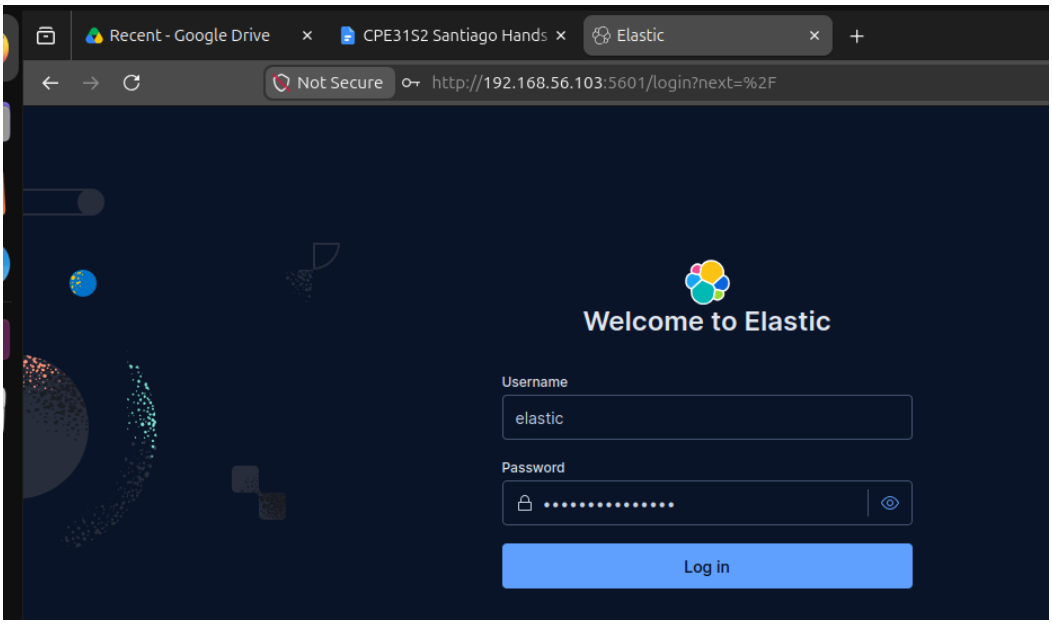
VERIFICATION OF ELK IN EACH HOST

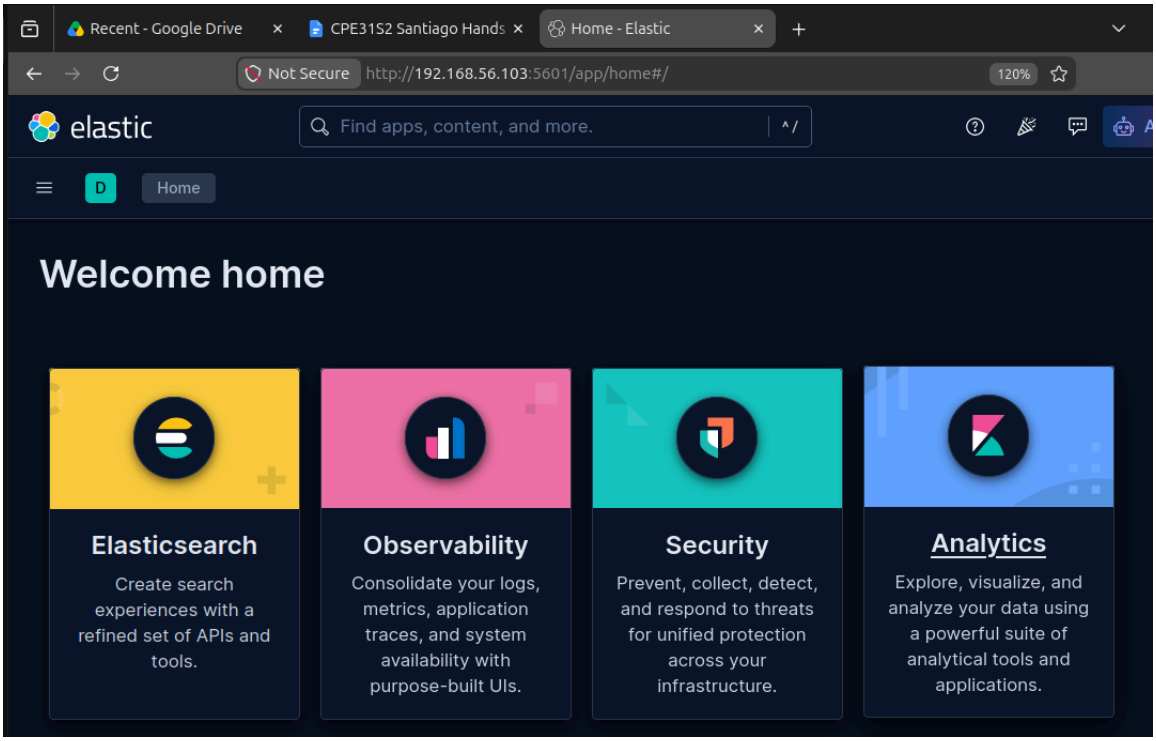
ElasticSearch (Ubuntu1 & Centos1)





Kibana (Ubuntu2)





Logstash (Ubuntu1):

```
server1@server1:~$ systemctl status logstash
● logstash.service - logstash
     Loaded: loaded (/usr/lib/systemd/system/logstash.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-10-13 00:04:23 PST; 12min ago
 Invocation: ab8155c65e974f3e9a2ce83aff2b06a3
   Main PID: 8402 (java)
      Tasks: 48 (limit: 4103)
     Memory: 648.5M (peak: 665.3M)
        CPU: 45.175s
     CGroup: /system.slice/logstash.service
             └─8402 /usr/share/logstash/jdk/bin/java -Xms1g -Xmx1g -Djava.awt.headless=true -Dfile.encoding=UTF-8

server1@server1:~$ ss -tlnp | grep 5044; curl -s localhost:9600; echo ""
LISTEN 0      4096                 *:5044             *:*
{"host":"server1","version":"9.5.5","http_address":"127.0.0.1:9600","id":"e06722a0-5535-49d4-908c-3a52648c622c","
name":"server1","ephemeral_id":"673aba7b-3bb9-456e-a53e-f5201f58a49d","snapshot":null,"status":"green","pipeline"
:{"workers":2,"batch_size":125,"batch_delay":50}}
server1@server1:~$
```

GITHUB REPO: https://github.com/owen-s-dotdev/CPE212-Santiago.git

DOCUMENTATIONS USED:

Elastic Search:

https://www.elastic.co/docs/deploy-manage/deploy/self-managed/install-elasticsearch-with-debian-package](https://www.elastic.co/docs/deploy-manage/deploy/self-managed/install-elasticsearch-with-debian-package#deb-key)
https://www.elastic.co/docs/deploy-manage/deploy/self-managed/install-elasticsearch-with-rpm
https://www.elastic.co/docs/deploy-manage/deploy/self-managed/important-settings-configuration

Kibana:
https://www.elastic.co/docs/deploy-manage/deploy/self-managed/install-kibana-with-debian-package
https://www.elastic.co/docs/deploy-manage/deploy/self-managed/install-kibana-with-rpm
https://www.elastic.co/docs/deploy-manage/deploy/self-managed/configure-kibana\

Logstash:
https://www.elastic.co/docs/reference/logstash/installing-logstash

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Reflections:
Answer the following:
1. What are the benefits of having a log monitoring tool?

Log monitoring tools allow us to track and monitor the status of each service in our managed systems. We can easily pinpoint issues, downtimes, and errors if they appear. Moreover, log files allow us to view the exact error logs which not only provides us key information for debugging but they also strengthen security; log files contain all connections and access times. As a system administrator, it is crucial that we have eyes everywhere and monitoring tools are the best options.

4. CONCLUSION/ANALYSIS/LEARNINGS

This activity took me a while to accomplish because of how many steps are required for each ELK component. I had to experiment and explore several ansible modules to actually implement the steps correctly since I cannot just inject the terminal commands into the playbooks. The requirement to install them separately per node also required me to be meticulous with each step. A lot of the issues I encountered were missing or incorrect configurations like assigning the wrong http port and forgetting to create a playbook to disable the firewall. Overall, this was an important learning experience especially since most of my time was spent troubleshooting.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Suggestion of ansible commands, assistance in verification, directory structure verification, debugging, ELK structure clarification.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Suggest alternative ansible commands when converting the documentation steps; clarification of how each ELK component communicates with each other; debugging connection issues and correct port assignments. My analysis and application of the concepts were entirely my own.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.